

Fase 3 – Projeto 6

SpaceHub – Conectando Lugares e Pessoas

O desafio que consolida sua formação

Este é o projeto de maior peso do seu portfólio.

O **SpaceHub** não existe para mostrar apenas que você sabe programar. Ele existe para provar que você consegue construir uma aplicação **ponta a ponta**, com regras de negócio reais, múltiplos tipos de usuário, persistência em banco de dados, autenticação, controle de disponibilidade e integração completa entre frontend e backend.

Aqui, você vai enfrentar um tipo de problema que aparece no mercado de verdade: **como garantir que várias pessoas possam interagir com os mesmos dados sem gerar conflito, perda de informação ou reserva duplicada.**

Quando você entrega uma plataforma que cadastra imóveis, lida com calendário, impede duas reservas no mesmo período e mantém tudo salvo em um banco real, você deixa de parecer apenas alguém que escreve código. Você passa a demonstrar capacidade de construir um produto utilizável por empresas do setor imobiliário, coworkings ou operações de gestão de espaços.

Posicionamento do projeto

O SpaceHub é um projeto de nível **avançado**, com foco em **Full-Stack**, envolvendo **frontend**, **backend** e **banco de dados**, com duração sugerida de **12 semanas**, ocupando a metade final da jornada de aprendizado.

A configuração pode ser **individual** ou **em dupla**, simulando um ambiente real de desenvolvimento colaborativo.

Dentro da progressão da formação, este projeto representa o ápice. Se na **Fase 1** o foco estava na tela, e na **Fase 2** na lógica local, aqui entra a memória permanente: servidor, banco de dados e múltiplos usuários compartilhando a mesma aplicação.

O SpaceHub exige integração total do que você aprendeu até agora. Você não está mais construindo uma peça isolada, mas uma solução robusta, com potencial real de uso por administradoras de imóveis, espaços de eventos ou redes de coworking.

O cenário que justifica a solução

O mercado de gestão de espaços, sejam eles apartamentos de temporada, salas de reunião ou estações de trabalho, lida com um problema clássico: **a informação descentralizada**.

Sem uma plataforma central, a operação depende de planilhas, e-mails, mensagens e confirmações manuais. Isso torna o processo lento, inseguro e sujeito a falhas graves, como o **overbooking**, quando duas reservas acabam aceitas para o mesmo período.

A dor principal aqui é a falta de sincronia.

Se um proprietário altera o valor da diária ou bloqueia uma data para manutenção, essa informação precisa chegar imediatamente à tela de qualquer possível cliente. Quando isso não acontece, o sistema deixa de inspirar confiança.

É exatamente aqui que o SpaceHub se torna relevante.

Seu papel será construir uma plataforma em que o dado é centralizado, a disponibilidade é confiável e a experiência do usuário transmite segurança no momento mais crítico: o clique em **Reservar**.

Quem está envolvido

Neste projeto, há dois perfis principais de uso.

O **anunciante**, ou **host**, precisa de um painel em que consiga cadastrar o imóvel, enviar fotos, definir preços e gerenciar o calendário de disponibilidade.

O **viajante ou profissional**, ou **guest**, busca velocidade e clareza. Ele quer encontrar espaços por localização, preço e características, mas acima de tudo precisa confiar que, ao reservar um período, aquele espaço realmente estará garantido.

Este é também o primeiro projeto em que você vai lidar com a complexidade de **persistência compartilhada**. Se o usuário A reserva um espaço entre os dias 10 e 15 de maio, o banco de dados precisa instruir a interface do usuário B a exibir esse mesmo período como indisponível.

Objetivo do projeto

Ao final das 12 semanas, você deverá entregar uma **aplicação Full-Stack funcional**, com autenticação de usuário, banco de dados e lógica de disponibilidade de datas.

O foco do projeto está no fluxo de dados entre o usuário, a API e o banco, com uma regra central inegociável: **não permitir reservas duplicadas para o mesmo espaço no mesmo período**.

Funcionalidades obrigatórias

Sua plataforma precisa entregar, no mínimo, as seguintes funcionalidades:

Autenticação e perfil

O sistema deve permitir **criação de conta** e **login seguro**, diferenciando dois papéis de uso:

- **Anfitrião**, que anuncia espaços
- **Hóspede**, que realiza reservas

Catálogo dinâmico

O anfitrião deve conseguir fazer o **CRUD completo dos imóveis**, podendo criar, ler, editar e excluir anúncios com:

- título
- descrição
- preço por noite
- fotos

Sistema de busca e filtro

O usuário deve conseguir buscar espaços por **localização** e aplicar filtros por características, como:

- com Wi-Fi
- sala de reunião
- apartamento inteiro

Motor de disponibilidade

A aplicação deve validar se o espaço está realmente livre nas datas escolhidas antes de permitir a reserva.

Fluxo de reserva

O sistema deve permitir solicitar uma reserva e salvar essa transação no banco de dados, vinculando o imóvel ao usuário e ao período selecionado.

O motor por trás da tela

Para sustentar essas funcionalidades, a arquitetura da aplicação precisa incluir:

- **API própria**, com rotas para processar reservas e consultas de imóveis
- **modelagem de banco de dados**, com tabelas relacionadas de **Usuários**, **Espaços** e **Reservas**
- **gestão de imagens**, para upload e exibição de fotos dos imóveis

- **segurança com tokens**, como **JWT**, impedindo que um usuário apague ou altere anúncios de outro

Como o projeto será construído

O cronograma está dividido em quatro marcos de três semanas, construindo a aplicação de dentro para fora.

Marco 1: A Fundação de Dados

Semanas 1 a 3

Você começa pela base estrutural do sistema.

O entregável é a modelagem do banco de dados relacional e a criação das primeiras rotas da API, começando pelo **CRUD de Espaços**.

O critério de sucesso é conseguir cadastrar um novo imóvel via **JSON** em uma ferramenta como **Postman**, e confirmar que o dado foi salvo corretamente no banco.

Marco 2: Segurança e Identidade

Semanas 4 a 6

Com a fundação pronta, você adiciona a camada de controle de acesso.

O entregável inclui cadastro e login de usuários com **JWT**, além de proteção de rotas para garantir que apenas o dono do anúncio possa editá-lo.

O critério de sucesso é simples e decisivo: ninguém consegue acessar o painel de **Meus Imóveis** sem estar autenticado.

Marco 3: O Motor de Reservas

Semanas 7 a 9

Agora entra a lógica mais sensível de todo o sistema.

O entregável desta fase é a implementação da disponibilidade, a criação da tabela de reservas e a integração da busca por data e local.

O critério de sucesso é que o sistema detecte conflitos de agenda e, ao tentar reservar um espaço em período já ocupado, retorne um erro claro e bloqueie a transação.

Marco 4: Interface e Lançamento

Semanas 10 a 12

Na etapa final, você conecta a interface ao backend e prepara a aplicação para o mundo real.

O entregável inclui:

- integração completa entre frontend e API
- upload de fotos dos imóveis
- deployment em servidor real

O checkpoint final exige que o usuário consiga abrir o site no celular, buscar um espaço, visualizar fotos e concluir uma reserva do início ao fim.

Conceitos fundamentais que você precisa dominar

Para transformar um site em uma plataforma dinâmica e persistente, você precisa entender como várias camadas trabalham juntas.

Arquitetura cliente-servidor

Você deve compreender o ciclo de vida de uma requisição HTTP: do clique no botão **Reservar** até a confirmação da operação no banco de dados.

APIs RESTful

Será necessário estruturar rotas de forma lógica, como:

- `POST /imoveis` para criar
- `GET /imoveis` para listar

Essa é a linguagem que conecta frontend e backend.

Modelagem de banco de dados

Você precisará definir como as entidades se relacionam. Por exemplo: como garantir que cada reserva esteja ligada a um único imóvel e a um único usuário.

Sistemas de autenticação com JWT

Será necessário entender como manter o usuário conectado sem reenviar a senha a cada ação, protegendo a privacidade e o acesso às rotas.

Operações CRUD

Você deve dominar o ciclo completo de criação, leitura, atualização e exclusão de dados, aplicando isso a imóveis, perfis e reservas.

Middlewares e validação

Também será necessário construir filtros no servidor para validar o que vem do usuário antes de salvar no banco, evitando inconsistências e falhas.

Estratégia de estudo com IA

Neste projeto, a IA pode ajudar principalmente em arquitetura, modelagem e depuração de backend, onde o feedback visual imediato nem sempre existe.

Modelagem de dados com IA

Antes de começar o banco, use a IA para validar sua lógica.

Prompt sugerido:

“Estou criando um sistema de reservas. Tenho as tabelas Usuários, Imóveis e Reservas. Como eu garanto que uma reserva esteja vinculada a um único imóvel e a um único usuário ao mesmo tempo? Gere um diagrama de texto (ERD) para mim.”

Depuração de rotas

Quando a API retornar um **500 Internal Server Error**, trabalhe com o log do terminal.

Estratégia:

cole o erro e a rota para a IA e peça:

“Analise este erro no meu arquivo de rotas e explique o fluxo de execução que levou à falha.”

Simulação de requisições

Use a IA para gerar objetos JSON de teste.

Prompt sugerido:

“Gere 5 exemplos de objetos JSON para cadastrar diferentes tipos de imóveis, como coworking, casa de praia e sala de reunião, para que eu possa testar minha API.”

Segurança e boas práticas

Pergunte sobre vulnerabilidades comuns:

“Como posso proteger minha rota de login contra ataques básicos? Explique o conceito de hash de senha e como implementá-lo.”

Documentação automática

A IA também pode ajudar a documentar o sistema, facilitando manutenção futura e clareza do README.

O que você deve aprender ao final deste projeto

Ao concluir o SpaceHub, você deverá demonstrar:

- **domínio de integração full-stack**, conectando interface, servidor e banco com fluidez e segurança
- **modelagem de dados relacionais**, representando corretamente relações entre usuários, imóveis e reservas
- **engenharia de APIs**, lidando com métodos HTTP e códigos de status de forma profissional
- **gestão de segurança e identidade**, controlando autenticação e autorização
- **manipulação de operações assíncronas**, com **Promises** e **async/await**
- **tratamento de arquivos e mídia**, recebendo, armazenando e exibindo imagens de imóveis
- **resiliência e validação de dados**, impedindo que entradas inválidas ou corrompidas contaminem o banco

Requisitos e restrições

Para que o SpaceHub funcione como software profissional, algumas regras são inegociáveis.

Requisitos funcionais

O sistema deve entregar:

- **sistema de contas**, com cadastro, login e logout seguro
- **gerenciamento de anúncios**, permitindo ao anfitrião criar, editar e excluir seus próprios espaços, incluindo upload de pelo menos uma foto
- **busca dinâmica**, com filtro por localização e verificação de disponibilidade
- **reserva com validação**, permitindo reserva apenas se o intervalo de datas estiver livre no banco
- **painel do hóspede**, onde o usuário visualiza reservas ativas e histórico

Requisitos técnicos

- uso obrigatório de **banco de dados real**, relacional ou não, para persistência permanente
- construção de **API própria**
- **criptografia obrigatória de senhas**
- **responsividade full-stack**, com fluxo de reserva funcional tanto no desktop quanto no celular

Restrições

É proibido:

- aceitar **double booking** sob qualquer circunstância
- usar dados **hardcoded** no HTML ou no JavaScript
- permitir que um usuário logado edite ou exclua anúncios de outra pessoa

- depender de **CMS** ou construtores sem código, como WordPress, Wix e similares

Recursos e documentação

Para construir a arquitetura full-stack do SpaceHub, você deverá se apoiar em ferramentas e referências que sustentem todo o fluxo entre usuário, API e banco.

Tecnologias de servidor

- **Node.js & Express**, para criação de rotas e tratamento do corpo das requisições
- **Postman** ou **Insomnia**, para validar a API antes da construção final da interface

Banco de dados e persistência

- **PostgreSQL** ou **MongoDB**, para estruturar imóveis, usuários e reservas
- **Cloudinary** ou **AWS S3**, para armazenamento das fotos, mantendo no banco apenas as URLs

Segurança e autenticação

- **JWT**, para sessões seguras
- **Bcrypt**, para hashing de senha antes do envio ao banco

Geolocalização e mapas

- **Leaflet.js** ou **Google Maps API**, para exibir os espaços em mapa interativo

Hospedagem

- **Render** ou **Railway**, para publicar frontend, backend e banco de dados em ambiente acessível por link público

Riscos e pontos de atenção

Neste projeto, os erros deixam de ser pequenos detalhes e passam a ser falhas de negócio, segurança ou integridade.

O pesadelo do overbooking

Se dois usuários tentarem reservar o mesmo espaço ao mesmo tempo, o banco precisa garantir consistência. A aplicação deve processar essas tentativas com cuidado, usando transações para assegurar que só uma reserva seja aceita.

Exposição de dados sensíveis

Nunca confie no que chega do navegador. Senhas em texto puro ou dados sensíveis expostos em respostas públicas da API comprometem toda a aplicação.

Acesso indevido (IDOR)

O servidor precisa verificar sempre se o recurso pertence ao usuário autenticado. Alterar um ID na URL ou no código não pode abrir brechas para editar ou excluir dados de terceiros.

Imagens pesadas e performance

Fotos muito grandes prejudicam toda a experiência de busca. O ideal é redimensionar e otimizar essas imagens para preservar fluidez.

Injeção de código

Campos de formulário podem ser usados para ataques se o servidor não tratar corretamente entradas maliciosas.

Queda do servidor por falta de tratamento

Toda rota precisa de **try/catch** e respostas controladas. Um erro não tratado não pode derrubar a aplicação inteira.

CrITÉRIOS de aceite

O SpaceHub só estará concluído quando resistir a um teste de uso real, simulando anfitriões e hóspedes utilizando a plataforma do início ao fim.

Checklist de validação

- **Fluxo de autenticação seguro:** o usuário cria conta, faz login e acessa áreas restritas; as senhas no banco estão criptografadas
- **Gestão de inventário (CRUD):** o anfitrião cadastra, edita e remove espaços com foto, e as mudanças aparecem na tela de busca
- **Busca com filtro de disponibilidade:** a pesquisa por data funciona corretamente e espaços indisponíveis não aparecem como livres
- **Prevenção de overbooking:** o servidor bloqueia reservas duplicadas para o mesmo imóvel e período, exibindo alerta claro
- **Persistência em nuvem:** banco de dados e API estão hospedados em serviço real, não apenas localmente
- **Performance de mídia:** as fotos carregam com rapidez no desktop e no celular
- **Tratamento de exceções:** o sistema exibe mensagens amigáveis ao receber dados inválidos, sem quebrar
- **Arquitetura limpa:** o código do servidor está organizado em rotas, controladores e modelos

Possibilidades de expansão

Depois de entregar o MVP, você pode transformar o SpaceHub em uma plataforma muito mais poderosa.

Simulação de pagamento

Integrar com uma API sandbox, como **Stripe** ou **PayPal**, para simular cobrança e gerar recibo em PDF.

Sistema de avaliações

Criar comentários e notas de 1 a 5 estrelas, com cálculo automático de média para reforçar confiança no anúncio.

Notificações em tempo real

Enviar e-mails automáticos com **Nodemailer** ou **SendGrid** para confirmar ou cancelar reservas.

Lista de desejos

Permitir que o usuário salve espaços favoritos em seu perfil.

Dashboard de ganhos

Criar gráficos com **Chart.js** para mostrar faturamento mensal e taxa de ocupação do anfitrião.

Busca por raio geográfico

Refinar a pesquisa para mostrar imóveis dentro de um raio específico da localização atual do usuário, usando GPS do celular.

Fechamento

O SpaceHub é o projeto em que você precisa provar que sabe lidar com dados compartilhados, conflitos de agenda, autenticação, persistência, mídia, integração e regras reais de negócio ao mesmo tempo.

Não é mais apenas sobre construir uma interface bonita ou uma lógica que roda localmente. É sobre entregar uma aplicação confiável, que coordena interesses de diferentes usuários e mantém a integridade da operação mesmo quando muita coisa acontece ao mesmo tempo.

Quando esse projeto estiver pronto, seu portfólio não estará apenas mais completo. Ele estará mais próximo do tipo de problema que empresas realmente pagam para resolver.